Notes on Grammatical Analysis

Sinan Olsson-Pasic

January 9, 2026

The following are a collection of notes that I'm documenting during my development and learning process of my grammatical analysis engine.

1 What is a language?

A language to a computer scientist is a possibly infinite set of sentences, each made up of "tokens" (words), and each token made up of "symbols" (letters). Or in other words, a language is a subset Σ^* for some alphabet Σ. Which "sentences" belong to a "language" and which "tokens" make up those "sentences" is explained by the definition of a generative grammar for the language.

A generative grammar is a 4-tuple (V_N, V_T, R, S) such that:

1. V_N and V_T are finite sets of symbols.
2. $V_N \cap V_T = \emptyset$
3. R is the set of pairs (P, Q) such that:
   (a) $P \in (V_N \cup V_T)^+$
   (b) $Q \in (V_N \cup V_T)^*$
4. $S \in V_N$

Where V_N is the set of non-terminals, V_T is the set of terminals, R is the set of rules, and S is the start symbol. $\emptyset$ denotes the empty set, meaning the intersection of the terminals and nonterminals returns no element.

A language L bound by a grammar G therefore is defined as:

$$L(G) = \{w \in V_t^* | S \to *w\} \tag{1}$$

All of this is to say that a generative grammar gives us the specification of how sentences for a language is constructed rather than listing the valid sentences. This allows us to describe infinite languages using finite information.

1.1 A note on infinite languages and descriptions

Is it possible to give a description for every language? The answer is no.